Special Participation E: Using ChatGPT to Automatically Find and Organize Related Papers

For this special participation, I designed an AI-enhanced prompt workflow using ChatGPT to help students deepen their understanding of lecture content by automatically finding and organizing related research papers.

The whole prompt workflow consists of three steps:

(1) Given a PDF of the lecture notes, the model extracts several core technical topics;

(2) For each topic, it generates specific search queries and uses them to retrieve relevant papers from Google Scholar or arXiv (ideally via **Agent Mode**);

(3) Based on the retrieved papers, it produces structured summaries that explicitly connect back to the lecture topics, proposes an appropriate 2-3 hour reading plan, and generates conceptual reflection questions. This turns paper reading activity into an intended, reflective and guided literature exploration rather than just paper searching and reading.

Here are prompts workflow I used:

Step 0: System prompt set up

```
"""
You are an AI assistant that helps graduate students deeply understand a specific lecture by:
(1) extracting the key technical topics from the given lecture notes,
(2) finding highly relevant research papers (especially classic or survey papers),
(3) summarizing those papers in a way that directly connects back to the lecture.

Your goals:
- Act as a substitute for pre-/post-lecture readings.
- Focus on conceptual clarity, not on generating fake citations.
- When you are uncertain about a paper or citation, explicitly say so instead of guessing.
"""
```

Step 1:

```
"""
I will give you pdf file from one lecture's notes (possibly noisy OCR).
Please:
```

```
1. Identify 5-10 core technical topics that this lecture is really about.
2. For each topic, list:
   - A short name (2-5 words)
   - 2-4 key phrases or keywords that would be useful for searching related papers
   - The pdf numbers or rough locations (if available) where this topic appears.

"""
```

Step 2 (In Agent Mode):

```
"""
Based on the topics you just extracted, please design literature search queries.
For each topic:
1. Generate 1-2 concrete search queries that can paste into Google Scholar / arXiv.
2. Make sure the queries are specific enough to avoid totally irrelevant fields, but not so narrow that
they return nothing.
3. Search in Google Scholar / arXiv using these queries.
4. For each query, find one or two highly relevant papers (preferably classic or survey papers) that seem
to directly address the topic.

Output as a markdown table, which contains generated queries and their target paper name, abstracts, and
URLs.
"""
```

Step 3:

```
"""
Now based on papers you searched.
For each paper, please:
1. Give a concise summary (3-5 sentences) focusing on:
   - The problem it studies
   - The main idea / method
   - The key result(s)

2. Explicitly connect it back to the lecture by answering:
   - Which topic(s) from this lecture does this paper directly relate to?
   - Which slide(s) or concept(s) would this paper help me understand better?
   - Does this paper generalize, formalize, or challenge something from the lecture?

3. Rate the paper for a student who has just seen this lecture:
   - "Good as a first paper to read"
   - "Better as a second/third paper"
   - "Too advanced unless doing a project"

4. Propose a reading plan for a student who:
```

```
   - has seen the lecture once,
   - wants to spend 2-3 hours doing deeper reading.

5. For each step, specify:
   - Which paper (or part of a paper) to read,
   - Which lecture concept it reinforces or deepens,
   - What question the student should keep in mind while reading.

6. End by listing 3-5 reflection questions that connect the lecture and the papers.
These questions should be conceptual (e.g., about assumptions, trade-offs, or generalization), not just
computational.

"""
```

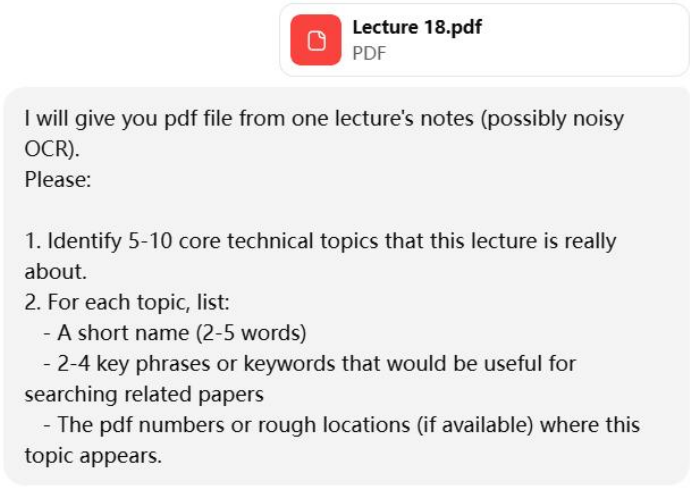
Actual Practice on Lecture 18 notes:

Here is the chat Link: https://chatgpt.com/share/6934b827-ed1c-8002-9a93-03242d44b324

Because the chat content is too long, I take first five topics as an example to finish the report.

Step 1:

Prompt



Response

✔ 5–10 Core Technical Topics

1) Encoder–Decoder with RNNs

- **Keywords:**
  `sequence-to-sequence`, `encoder decoder`, `translation`, `autoregressive`
- **Where:**
  Page 1: diagrams of RNN encoder → decoder
  Page 2: autoregressive generation, teacher forcing

---

2) Bottleneck in Seq2Seq

- **Keywords:**
  `fixed-length bottleneck`, `context vector`, `vanishing info`, `long sequences`
- **Where:**
  Page 2: "Need to capture all of the input sentence in this bottleneck."

---

3) Teacher Forcing

- **Keywords:**
  `teacher forcing`, `training technique`, `auto-regressive`
- **Where:**
  Page 2: training diagram with `<start>` and target tokens

---

4) Sampling and Output Generation

- **Keywords:**
  `sampling`, `probabilities`, `token generation`, `softmax`
- **Where:**
  Page 3: language example, probabilities for words

---

5) Motivation for Attention

- **Keywords:**
  `bottleneck`, `context`, `skip connections`, `U-Net`
- **Where:**
  Page 4: "Where did we see a bottleneck before? U-nets."

These topics may be too specific to become a 'TOPIC', but works well in following paper search step.

I love the summary part which is not contain in my prompt. Only topics look not very clear. However, with this summary part, it is easy to know what this lecture notes focus on.

📌 Summary of Core Lecture Themes

This lecture transitions from:

- **RNN encoder–decoder sequence models**
  →
- **Attention as a differentiable lookup mechanism**
  →
- **Self-attention + multi-head architectures**, the foundation of Transformers.

The **central pedagogical arc**:

1. RNN seq2seq **forces information into a bottleneck** (context).
2. We need a way to **"look back" at all tokens** → attention.
3. Attention = **similarity + weighted combination** via Q/K/V.
4. **Scaling and softmax** fix numerical stability.
5. **Multi-head and multi-query** improve expressiveness and memory.

Step 2:

Prompt

I didn't use Agent mode at first, GPT only searched a few papers, which are the most famous ones such as *Attention is all you need*. This is not what I want, so I changed into Agent Mode and turned out to be better.

Based on the topics you just extracted, please design literature search queries.

For each topic:
1. Generate 1-2 concrete search queries that can paste into Google Scholar / arXiv.
2. Make sure the queries are specific enough to avoid totally irrelevant fields, but not so narrow that they return nothing.
3. Search in Google Scholar / arXiv using these queries.
4. For each query, find one or two highly relevant papers (preferably classic or survey papers) that seem to directly address the topic.

Output as a markdown table, which contains generated queries and their target paper name, abstracts, and URLs.

Response

Lecture topic & purpose	Search query (Google Scholar/arXiv)	Relevant paper(s) and short summary	URL
1. Encoder–Decoder with RNNs (introducing sequence-to-sequence models for tasks like translation)	`sequence-to-sequence RNN encoder decoder translation neural network`	*Sutskever et al., "Sequence to Sequence Learning with Neural Networks"* – uses stacked LSTMs to map an input sequence to a fixed-length vector and then decode to an output sequence; demonstrates strong English–French translation performance and notes that reversing source sentences helps optimization arxiv.org .	https://arxiv.org/abs/1409.3215 ↗
		Cho et al./Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate" – extends seq2seq by introducing an attention mechanism that soft-selects source positions; it addresses the fixed-length bottleneck and achieves performance comparable to phrase-based translation systems arxiv.org .	https://arxiv.org/abs/1409.0473 ↗
2. Bottleneck in Seq2Seq (understanding the limitations of fixed-length context vectors)	`fixed-length context vector bottleneck seq2seq "encoder-decoder"`	*Bahdanau et al., 2014* (above) – argues that encoding a whole sentence into a single vector is a bottleneck and introduces attention to allow the decoder to focus on different source words arxiv.org .	see above
		Vaswani et al., "Attention Is All You Need" – shows that an attention-only architecture (the Transformer) outperforms recurrent encoder-decoders and eliminates the bottleneck by using self-attention and cross-attention layers arxiv.org .	https://arxiv.org/abs/1706.03762 ↗
3. Teacher Forcing (training technique for RNNs)	`teacher forcing recurrent network training algorithm`	*Lamb et al., "Professor Forcing: A New Algorithm for Training Recurrent Networks"* – describes teacher forcing (feeding ground-truth tokens during training) and proposes an adversarial method (Professor Forcing) that matches the network's dynamics during training and sampling; this improves sequence generation quality arxiv.org .	https://arxiv.org/abs/1610.09038 ↗
4. Sampling and Output Generation (decoding strategies for language models)	`neural text degeneration nucleus sampling decoding`	*Holtzman et al., "The Curious Case of Neural Text Degeneration"* – observes that maximizing likelihood during decoding leads to bland, repetitive text; proposes nucleus (top-p) sampling to draw from the "dynamic nucleus" of the probability distribution, yielding more diverse yet fluent outputs arxiv.org .	https://arxiv.org/abs/1904.09751 ↗
5. Motivation for Attention / Skip Connections (connecting U-Net bottlenecks to attention)	`U-Net encoder-decoder skip connections attention bottleneck`	*Ronneberger et al., "U-Net: Convolutional Networks for Biomedical Image Segmentation"* – introduces an encoder-decoder network with a contracting path and a symmetric expanding path linked by skip connections; the architecture captures context and enables precise localization arxiv.org . Though not about language, U-Net illustrates how skip connections alleviate information bottlenecks.	https://arxiv.org/abs/1505.04597 ↗

It is totally acceptable to do this because some topics are closely related

Step 3:

Prompt

After papers have been found, this step is relatively easy to accomplish. You can also change your own prompt to do things you want. Generating reflection questions helps me better understand the relationship between lecture notes and papers. This is important for me.

Now based on papers you searched.
For each paper, please:

1. Give a concise summary (3-5 sentences) focusing on:
   - The problem it studies
   - The main idea / method
   - The key result(s)

2. Explicitly connect it back to the lecture by answering:
   - Which topic(s) from this lecture does this paper directly relate to?
   - Which slide(s) or concept(s) would this paper help me understand better?
   - Does this paper generalize, formalize, or challenge something from the lecture?

3. Rate the paper for a student who has just seen this lecture:
   - "Good as a first paper to read"
   - "Better as a second/third paper"
   - "Too advanced unless doing a project"

4. Propose a reading plan for a student who:
   - has seen the lecture once,
   - wants to spend 2-3 hours doing deeper reading.

5. For each step, specify:
   - Which paper (or part of a paper) to read,
   - Which lecture concept it reinforces or deepens,
   - What question the student should keep in mind while reading.

6. End by listing 3-5 reflection questions that connect the lecture and the papers.
These questions should be conceptual (e.g., about assumptions, trade-offs, or generalization), not just computational.

Response

It gave 10 paper as a whole, but in this report I take first two paper as an example.

Nice, this is exactly the kind of "mini reading course around one lecture" that this tool is meant to be. I'll focus on the **most relevant papers for this lecture**:

- Seq2Seq + bottleneck + attention:
  1. Sutskever et al. 2014 – *Sequence to Sequence Learning with Neural Networks*
  2. Bahdanau et al. 2014 – *Neural Machine Translation by Jointly Learning to Align and Translate*
  3. Vaswani et al. 2017 – *Attention Is All You Need*
- Training / decoding behavior:
  4. Lamb et al. 2016 – *Professor Forcing*
  5. Holtzman et al. 2019 – *The Curious Case of Neural Text Degeneration*
- Efficient attention variants:
  6. Shazeer 2019 – *Fast Transformer Decoding: One Write-Head is All You Need*
  7. Ainslie et al. 2023 – *GQA: Training Generalized Multi-Query Transformer Models...*
- Conceptual analogies / memory:
  8. Ronneberger et al. 2015 – *U-Net*
  9. Graves et al. 2014 – *Neural Turing Machines*
  10. Weston et al. 2014 – *Memory Networks*

I'll go paper by paper, then give you a concrete **2–3 hour reading plan**, then some **reflection questions**.

1. Sutskever et al. 2014 – Sequence to Sequence Learning with Neural Networks

1. Concise summary (3–5 sentences)
This paper studies how to map one sequence (e.g., a source sentence) to another (e.g., a translation) using neural networks. The main idea is an **encoder–decoder RNN**: a deep LSTM encodes the source into a fixed-length vector, and another LSTM decodes that vector into the target sequence. They find tricks like **reversing the source sentence** improve performance and show competitive results on large-scale machine translation. The key result is that pure neural sequence models can work surprisingly well for translation, without explicit phrase tables.

2. Connection to the lecture

- **Lecture topics:**
  – Encoder–Decoder with RNNs
  – Bottleneck in Seq2Seq (fixed context vector)
- **Helps with:**
  – The early slides where you see an RNN encoder feeding a single "context vector" into a decoder.
  – Understanding *why* the lecture keeps complaining about the "bottleneck".
- **Role w.r.t lecture:**
  – This paper **formalizes and exemplifies** the vanilla encoder–decoder model that the lecture presents as the starting point.
  – It does *not* solve the bottleneck; it is exactly the thing Bahdanau/attention is reacting to.

3. Rating for a student after this lecture
Good as a first paper to read. Dense but conceptually straightforward once you've seen the lecture diagrams.

This rating is actually reading order for us, which is important. I think it is even more important than summary part.

2. Bahdanau et al. 2014 – Neural Machine Translation by Jointly Learning to Align and Translate

1. Concise summary
This paper targets the **fixed-length bottleneck** in encoder–decoder models for translation. The core idea is **additive (Bahdanau) attention**: instead of compressing the source into one vector, the decoder learns to attend to different source positions at each step via a learned alignment mechanism. The method computes alignment scores between the current decoder state and each encoder state, normalizes them, and forms a weighted sum as a dynamic context. The key result is improved translation quality and interpretable "soft alignments" between source and target words.

2. Connection to the lecture

- **Lecture topics:**
  – Bottleneck in Seq2Seq
  – Attention as differentiable matching
  – Query/Key/Value (early form)
  – Cross-attention encoder–decoder
- **Helps with:**
  – The slide "Need to capture all of the input sentence in this bottleneck" → Bahdanau shows you the fix.
  – The diagrams where the decoder "looks back" at all encoder states with attention weights.
- **Role w.r.t lecture:**
  – This paper **directly generalizes and fixes** the basic RNN encoder–decoder from the lecture.
  – It provides the **historical bridge** between RNN Seq2Seq and the more general attention-as-lookup view.

3. Rating
Excellent as a first/second paper to read. It's basically the "attention motivation" paper.

2–3 Hour Reading Plan (Practical schedule)

Assume you've seen the lecture once and have 2–3 hours. Here's a **concrete path** with steps, what to read, what it reinforces, and a guiding question.

Step 1 (25–30 min) – From RNN Seq2Seq to the Bottleneck

- **Paper & sections:**
  – *Sutskever et al. 2014*, Abstract + Intro + Model description (the core encoder–decoder idea; you can skim experiments).
- **Lecture concepts reinforced:**
  – Encoder–decoder with RNNs, fixed context vector, autoregressive decoding.
- **Guiding question:**

> *If the encoder has to compress the whole sentence into one vector, what kinds of inputs are likely to be hard for this model?*

In my practice, reading a paper without reflective questions is not effective

---

Step 2 (35–40 min) – How Attention Fixes the Bottleneck

- **Paper & sections:**
  – *Bahdanau et al. 2014*, Abstract + Intro + Model (especially the alignment / attention mechanism) + the alignment visualizations.
- **Lecture concepts reinforced:**
  – Bottleneck, cross-attention, attention as soft alignment / weighted average over encoder states.
- **Guiding question:**

> *How exactly does the attention mechanism change what information the decoder has access to at each time step, compared to Sutske ↓ 's model?*

Step 3 (40–45 min) – Attention as a Full Architecture (Transformer)

- **Paper & sections:**
  – *Vaswani et al. 2017*, Abstract + Sections 1–3 (background, attention, and overall Transformer architecture). Skim later experimental sections.
- **Lecture concepts reinforced:**
  – Query/Key/Value, scaled dot-product attention, self-attention vs cross-attention, multi-head attention.
- **Guiding question:**

> *Why does replacing recurrence with multi-head self-attention make training more parallelizable, and how does multi-head attention generalize the Bahdanau-style mechanism?*

Step 4 (30–35 min) – Sampling & Training Behaviors

Split this into two short, conceptually lighter reads.

1. **Holtzman et al. 2019 (The Curious Case of Neural Text Degeneration)**
   - **Sections:** Abstract + Intro + experiments describing degeneration and nucleus sampling.
   - **Concepts reinforced:** sampling from softmax, decoding strategies, why greedy/beam ≠ "best text".
   - **Guiding question:**

     > *How can the same model produce very different text quality just by changing the decoding procedure?*

2. **Lamb et al. 2016 (Professor Forcing)**
   - **Sections:** Abstract + method overview (you can skim technical details of the adversarial objective).
   - **Concepts reinforced:** teacher forcing, difference between training and sampling regimes.
   - **Guiding question:**

     > *What is the mismatch between training with teacher forcing and test-time sampling, and how does Professor Forcing try to fix it?*

This step ties together the lecture's "training vs inference" and "sampling from probabilities" into more realistic generation behavior.

Step 5 (Optional 20–30 min, if time) – Efficient Multi-Head Variants

- **Paper & sections:**
  – *Shazeer 2019*, Abstract + main idea (multi-query attention) + experiments.
- **Concepts reinforced:**
  – Multi-head attention costs, multi-query variant, memory bandwidth.
- **Guiding question:**

> *If every head has its own K/V, why is that expensive in practice, and what do we lose by forcing all heads to share K/V (multi-query)?*

If you're very motivated and still have time, skim the **GQA** paper's abstract & method diagram to see how multi-query generalizes to grouped-query.

Reflection Questions (3–5 conceptual prompts)

Use these after you've done the reading; they're meant to tie the lecture and papers together.

1. **Bottleneck vs Attention:**
   - In Sutskever vs Bahdanau, how does the type of bottleneck change, and what does that imply about the *length* and *structure* of sequences each model can handle well?
2. **What is Attention Really Doing?**
   - If you think of attention as a differentiable hash table / memory lookup, what "keys" and "values" are being stored in the encoder states, and how does the query vector decide what is relevant?
3. **Multi-Head vs Multi-Query Trade-off:**
   - Conceptually, what kind of patterns can multi-head attention represent that single-head (or multi-query) attention might struggle with? When is it worth sacrificing some of that expressiveness to save memory and compute?
4. **Training vs Inference Mismatch:**
   - Teacher forcing, Professor Forcing, and nucleus sampling all deal with mismatches between the training objective and how we use the model at test time. How are these mismatches different (exposure bias vs decoding objective), and which seems more fundamental?
5. **From RNNs to Transformers:**
   - After reading these papers, do you see the Transformer as a completely new paradigm, or as a natural extension of encoder–decoder RNNs with attention and memory? In what ways is it "just" stacking and refining the same attention idea, and in what ways is it qualitatively different?